

Python System Documentation

Carr+

AxelP-1

May 2026

Contents

0.1	Introduction	2
0.2	Dependencies	2
0.3	Use	2
1	Installation	2
2	Usage and functions	2
2.1	waveFolder	3
2.2	Karplus Strong algorithm	3
2.3	varispeed	3
2.4	DJ stutter	4
2.5	cutToBeats	4
2.6	Automatable filter	4
2.7	Synth	4
2.8	Vocoder	5
2.9	Utilities	5
2.10	Additive synth	5
2.11	Default synthesized clones of instruments	5
2.12	Hz to semitones	6
2.13	Artificial double tracking	6
2.14	Pedal board	6
2.15	Drum machine	7
2.16	Reverb	8
3	Troubleshooting	8
4	Conclusion	8

0.1 Introduction

Carr+ is a python based system for audio generation that leaves lots of versatility, and lets you see how it works, making for hackable DSP.

0.2 Dependencies

The following things are needed

- Python 3.x
- NumPy
- Math
- Sympy
- matplotlib.pyplot
- scipy
- IPython.display

0.3 Use

Use can be done in any python system but for ease of use, google colab or a similar system is recommended so that function definition isn't run every time a function is used.

1 Installation

To use, go to [github source page](#) and copy the code for use, pasting it into your python editor.

2 Usage and functions

There are two main types of prebuilt systems. Classes or functions. A class includes functions used together often and must be initialised before use. For example, with the Karplus class:

```
#Define the karplus strong paramters
k=Karplus(1)
#Generate the note 440Hz, A4
note=k.KarplusGenerate(440)
#Use the same settings to generate another note
note2=k.KarplusGenerate(200)
```

2.1 waveFolder

A wavefolder is a type of distortion. [Wikipedia link](#)

```
waveFolder(vals, foldfactor=1.3, mult=4, speedOfFolding=2,
           sinAmplitude=0.2)
```

The parameters are vals, an array or list of values, foldFactor, how much the folding dies out as the amplitude increases, limiting how much folding can be done, foldFactor of 1 means the folding never dies out while 2 means that an increase of 1 in amplitude is a division by 2 in fold size, mult is gain before entering the wavefolder, speedOfFolding is how many folds are generated for a fixed amount of amplitude, sinAmplitude is how large the folds are.

2.2 Karplus Strong algorithm

Karplus is a string synthesis system. [Wikipedia link](#) The implementation of Karplus Strong uses a class to group functions.

```
Karplus(bufferSize, attack=0.1, sampleRate=44100, release=0.1)
```

The parameters are bufferSize, the size of the buffer in seconds, must be longer than one divided by the lowest frequency used, attack, how fast the noise's attack is in seconds, creates corresponding attack in the final note, sampleRate is the sample rate and release is how much the final output is faded out.

```
delay(self, samples, freq, decayPerSecond, hiCut)
```

The parameters are samples, the input, freq, how many times per second the system echoes, decayPerSecond is how much the signal is divided every second, hiCut is how much frequencies are attenuated, value needs to be very large, of size about 1000000000000.

```
shortNoiseBurst()
```

The function has no parameters, as they are previously defined in the initialization of karplus strong.

```
KarplusGenerate(Hz, decay=2, length=2, hiCut=1000000000000)
```

The function to synthesize the string takes a frequency input in Hz, decay which is how much the amplitude is divided every second, length is how long the note lasts and hiCut is how much higher frequencies are attenuated. As frequencies vary very little, a large multiplier is needed.

2.3 varispeed

Varispeed is an effect where the playback speed of a recording is changed or varies from the intended speed. This implementation uses lets you automate speeds. [Wikipedia link](#)

```
rhythmicVarispeed(samples, speeds, times, startTime=0, sr=44100)
```

The function lets you input samples, then a speeds list, which includes various speeds at which the system will have the samples played and times is when the sample will be played back at that speed. The system assumes initial speed is 1 unless overridden. The startTime tells the system when the tapehead should start and sr is the sample rate. For example, for a tape stop at second number 7 of a recording, we can do the following:

```
rhythmicVarispeed(recording,[1,0],[7,7.5],startTime=0,sr=44100)
```

Which means that as we don't specify a speed at time=0, it starts off at a speed of 1 and the speed stays the same until the seventh second where it starts going down, to reach speed 0 at t=7.5. The speeds can be negative or positive. The speeds list and times list should be the same size. The recording should be long enough otherwise an error will occur.

2.4 DJ stutter

Stutter is an effect where a block of audio is played multiple times creating a glitch like effect. [Wikipedia link](#)

```
stutter(sample,start, reps,duration,sr=44100)
```

Sample is the audio the effect should be applied to, start is when it starts, reps is how many times the block is repeated and duration is how long is seconds the block is. sr is sample rate.

2.5 cutToBeats

Cuts audio to a given length in beats.

2.6 Automatable filter

A filter whose cutoff can be automated in a similar way as the varispeed, using two lists. [Wikipedia link](#)

```
filterWithAutomation(data,hicut,times,sr=44100,order=5)
```

data is the data to be filtered. hicut is a list of filter cutoffs, and times is when each of these cutoffs is applied. sr is the sample rate and order is how much the filter cuts off. After the defined cutoffs, it assumes constant cutoff.

2.7 Synth

A sound synthesis system. [Wikipedia link](#) It is implemented as a class.

```
synth(sr,wavetype,a,d,s,r,bpm)
```

sr is the sample rate, wavetype is the type of oscillator used, either saw, sine or square, a d s and r are the standard envelope parameters except r is a fade out, and bpm is the bpm used for the synth.

```
def synthesizeNote(note,beats,volume=1,pw=0.5)
```

note is the note as a string as scientific pitch notation so "A4" is 440, volume is how loud the note is played and pw is the pulse width.

2.8 Vocoder

A vocoder for musical use, implemented as a class. [Wikipedia link](#) It was implemented as a class so that you can switch the modulator or carrier but keep the same settings.

```
vocoder(carrier, modulator, samplerate, bands, fmn, fmx, smear, order)
```

Carrier is the carrier, modulator is the modulator, samplerate is the sample rate, bands is how many bands are used, fmn is the lowest frequency kept and fmx is the highest, smear is how much the envelope is averaged, in seconds, order is how much the frequencies outside of a band are filtered out, each order is 6dB/Oct.

```
vocode()
```

vocodes, using the modulator, carrier and predefined parameters.

```
#To change the modulator
vcd.modulator=audio
#To change the carrier
vcd.carrier=audio
```

2.9 Utilities

```
taper(arr, start, end)
```

Tapers beginning of list for avoiding clicking where splicing was done or to generate envelope.

```
pad(arr, d)
```

Adds d zeroes before and after an array.

```
unpad(arr)
```

Removes zeroes from edges of list.

2.10 Additive synth

Generates an additive waveform. [Wikipedia link](#)

```
customSynth(freq, sampleRate, leng, harmonics)
```

freq is the frequency of the note played, leng how long it is played for, sampleRate is the sample rate, harmonics is a list of tuples including how many times the base frequency it is and how much of it you add.

2.11 Default synthesized clones of instruments

These are preset clones of instruments.

```
piano(freq, sampleRate, leng)
```

freq is the frequency, sampleRate is the sampleRate, leng is the length in seconds.

```
harmonica(freq, sampleRate, leng)
```

Same parameters as piano.

```
violin(freq, sampleRate, leng)
```

Same parameters as piano

```
flute(freq, sampleRate, leng)
```

Same parameters as piano.

2.12 Hz to semitones

```
noteToFreq(note)
```

Converts a note string to Hz.

2.13 Artificial double tracking

Creates a delayed copy of audio and adds the two together to get a double tracking like effect. [Wikipedia link](#)

```
adt(arr, d)
```

Double tracks with a delay of d samples on the copy.

```
revAdt(arr, d)
```

Removes an artificial double tracking of d samples, generates weird effect if d is off or no double tracking occurred.

2.14 Pedal board

A set of effects similar to a pedal board. [Wikipedia link](#) The use of the PedalSystem class is particular, as some functions need the time to function. Additionally, it is done using blocks. For use, the blocks can be of size 1 as it is mostly used offline but for faster results, a larger size may be needed. Below is an example usage of the class, using two effects. A buffer is only needed for certain functions but should always be used. The blocks must of a size less than 500.

```
pedals=PedalSystem(sampleRate=44100, rllvrStp=3, maxLen=4)
output=[]
for i in audio:
    temp=pedals.tremolo([i],0.5,5)
    temp=pedals.overdrive(temp,3)
    pedals.count(temp)
```

```
PedalSystem(sampleRate=44100, rllvrStp=3, maxLen=4)
```

sampleRate is the sample rate, rllvrStp is the standard steepness for overdrive, can be overridden, maxLen is the buffer length.

Overflow is a soft clipping function using tanh for values above the maximum clean range.

```
overdrive(sample, multiplier, maxVal=1, rolloverSteepness=-1)
```

maxVal is the maximum amplitude before distortion, sample is an input, multiplier is the gain and rolloverSteepness is how steep the rollover is, with -1 being the default defined and infinity being a brick wall.

Count is a function used to ensure that no two functions increment the time at the same time, making the time counter speed up. It also fills the buffer.

```
count(samples)
```

The only input is the samples that it should add to the buffer.

```
tremolo(sample, depth, speed)
```

Applies tremolo on a list of samples, sample. Depth is how much the amplitude is modulated, 0 being not at all, 1 being going from no sound to twice the normal amplitude. Speed is how often it does this, in Hz.

```
vibrato(self, sample, depth=0.005, speed=5)
```

Vibrato has the following parameters: sample, the input audio, depth, the depth of the modulation and speed, how many times per second it does it.

```
delay(sample, time, divisionFactor=2)
```

Adds a delay to audio, sample is the input audio, time is how long it takes to echo back and divisionFactor is how much each echo is divided.

```
noiseGate(sample, depth)
```

Creates a noise gate, muting everything below depth.

```
bitcrusher(sample, bits)
```

Crushes the bit depth of sample to bits, assuming a maximum amplitude of 1.

2.15 Drum machine

A set of individual functions used to generate drum-like sounds. [Wikipedia link](#)

```
snare(pitch, pitchMix, noiseburstFreqMin, noiseburstFreqMax, top, fall, decay, attackSine, decaySine, time, sr=44100, order=5)
```

pitch is the pitch of the sine wave, pitchMix is how loud compared to the sustain of the noise the sine wave is, noiseburstFreqMin is the lowest frequency not attenuated, noiseburstFreqMax is the highest frequency not attenuated, top is how many times louder than the sustain the initial crack of the snare is, fall is how long it takes after the crack of the snare to reach the sustain, attackSine is the attack of the sine and decay sine is the decay of the sine, time is the length of the whole snare, sr is the sample rate and order is the butter worth filter's order.

```
hiHat(noiseburstFreqMin, noiseburstFreqMax, attack, decay, time, sr=44100, order=5)
```

Similar parameters as for snare, noiseburstFreqMin is the lowest unattenuated frequency, noiseburstFreqMax is the highest unattenuated frequency, attack is the attack time and decay is the decay time, time is the length of the note, sr is the sample rate and order is the order of the filter, each order being -6dB/Oct.

```
kick(freqStart , freqEnd , fmFreq , fmAmount , attack , decay , time ,
    timeAfterSweep , grit=2 , sr=44100)
```

The kick is a slide in frequency, whose parameters are freqStart, the frequency at which it starts, freqEnd, the frequency at which it stops, fmFreq, the frequency of the modulator for FM to generate grittier tones, fmAmount, how much FM there is, attack, the attack of the kick, decay, the decay of the kick, time, how long the kick takes to reach the endFreq and timeAfterSweep, how long it stays at freqEnd, grit, which is how much the kick is wavefolded to generate harsher tones and finally, sr, sample rate.

2.16 Reverb

Reverb gives a sense of space. This uses primes to avoid comb filtering.

```
applyReverb(signal , minDelay , maxDelay , channels , blocks , mix , loCut ,
    hiCut , sr=44100)
```

Signal is the input signal, minDelay is the minimum time for it to start, maxDelay is the reverb time, channels is how many reverb channels there are, blocks is how many diffusion blocks are applied, mix is the mix for the reverb, loCut and hiCut are the low and hi cut frequencies and sr is the sample rate.

3 Troubleshooting

Outputs are given as lists generally.

4 Conclusion

The system can be improved so feel free to read through and change things.